

BASES DE DATOS

Práctica 1: "Modelo Entidad Relación"

Documento de investigación

Nombre: Serna Villa Jurgen Andrey

Grupo: 3CV4

Boleta: 2025630361

Fecha: 18 de septiembre de 2026

Índice

1. Ejercicio 1. Control de versiones con Git y GitHub
2. Ejercicio 2. El sistema gestor en un contenedor: Docker
3. Ejercicio 3. Investigación: qué es una base de datos
4. Ejercicio 4. Estado del arte: tres artículos científicos
5. Referencias generales

Ejercicio 1. Control de versiones con Git y GitHub

Parte A. Investigación

1. ¿Qué es un sistema de control de versiones y qué problema concreto resuelve en un trabajo en equipo?

Un sistema de control de versiones es una herramienta que registra y organiza los cambios realizados sobre archivos a lo largo del tiempo. Permite conservar diferentes versiones de un proyecto, identificar qué modificaciones se realizaron, cuándo ocurrieron y, dependiendo de la herramienta y del flujo utilizado, quién las realizó. Git es un sistema de control de versiones distribuido: cada copia de un repositorio puede contener el historial necesario para trabajar y sincronizar cambios.

En un trabajo en equipo, el problema principal que resuelve es la coordinación de cambios. Sin control de versiones, varias personas pueden editar los mismos archivos y terminar sobrescribiendo trabajo, perdiendo modificaciones o sin poder determinar cuál versión es la correcta. Git permite crear ramas para desarrollar funcionalidades de manera aislada, registrar cambios mediante commits y fusionar posteriormente las modificaciones. También facilita regresar a una versión anterior cuando un cambio introduce un problema.

Por ejemplo, si tres integrantes desarrollan una aplicación y cada uno trabaja en una funcionalidad distinta, pueden crear ramas separadas. Cada integrante registra sus avances con commits y posteriormente las ramas se integran en la rama principal. Si aparece un error, el historial permite localizar el cambio relacionado y recuperar una versión estable.

2. Diferencia entre Git y GitHub

Git y GitHub no son sinónimos. Git es el sistema de control de versiones que se instala y ejecuta localmente para registrar cambios, crear ramas, consultar historial y realizar fusiones. Git funciona aunque no exista una cuenta de GitHub.

GitHub es una plataforma de alojamiento y colaboración alrededor de repositorios Git. Permite almacenar repositorios remotos, trabajar con Pull Requests, revisar código, administrar incidencias, configurar permisos y automatizar procesos. Por tanto, Git proporciona el mecanismo de control de versiones, mientras que GitHub proporciona servicios para alojar y colaborar sobre proyectos que utilizan Git.

Un ejemplo sencillo es el siguiente: un estudiante modifica un archivo en su computadora y ejecuta `git commit`; Git registra ese cambio en el repositorio local. Después puede utilizar `git push` para enviar la rama a un repositorio remoto alojado en GitHub. Otro integrante puede obtener los cambios mediante `git pull` o `git fetch` y trabajar sobre ellos.

3. Conceptos fundamentales

Repositorio: Espacio donde se almacena un proyecto y su historial de versiones. Puede ser local o remoto.

Confirmación o commit: Registro permanente de un conjunto de cambios en el historial. Un buen commit describe qué cambio se realizó y debe representar una unidad lógica de trabajo.

Rama o branch: Línea independiente de desarrollo que permite realizar cambios sin modificar directamente otra rama. La rama principal suele contener una versión integrada del proyecto.

Fusión o merge: Operación mediante la cual se incorporan los cambios de una rama en otra. Git intenta combinar automáticamente los cambios; si dos modificaciones son incompatibles, puede aparecer un conflicto que debe resolverse manualmente.

Conflicto de fusión: Situación en la que Git no puede decidir automáticamente qué versión conservar, normalmente porque dos ramas modificaron la misma parte de un archivo de formas incompatibles.

Pull Request: Mecanismo de colaboración de plataformas como GitHub mediante el cual se propone integrar los cambios de una rama en otra. Permite revisar código, discutir modificaciones, ejecutar comprobaciones y posteriormente fusionar el trabajo.

.gitignore: Archivo que especifica archivos o directorios que Git no debe incluir en el seguimiento, por ejemplo archivos temporales, credenciales, compilaciones o configuraciones locales.

README: Documento que explica el propósito del repositorio y proporciona información útil para instalar, ejecutar o comprender el proyecto.

Ejemplo de flujo con ramas

```
git clone <URL-del-repositorio>
git switch -c nueva-funcionalidad
git add .
git commit -m "Agrega validación de usuarios"
git push -u origin nueva-funcionalidad
```

Después se abre un Pull Request en GitHub para revisar la rama nueva y, una vez aprobada, fusionarla con la rama principal. Este flujo permite que el código sea revisado antes de integrarse.

4. Flujo de trabajo basado en ramas y revisión entre pares

Un flujo basado en ramas consiste en mantener una rama principal para el trabajo integrado y crear ramas específicas para nuevas funcionalidades, correcciones o documentación. Cada cambio se desarrolla de manera aislada, se registra mediante commits y posteriormente se propone para integración.

La revisión entre pares antes de la fusión permite detectar errores, problemas de diseño, código duplicado, vulnerabilidades o incumplimientos de los requisitos antes de que lleguen a la rama principal. Además, la revisión distribuye el conocimiento del proyecto entre los integrantes y deja un registro de las decisiones tomadas.

Un flujo básico es: actualizar la rama principal, crear una rama de trabajo, realizar cambios pequeños y descriptivos, ejecutar pruebas, publicar la rama, abrir un Pull Request, solicitar revisión, corregir observaciones y finalmente fusionar la rama.

Referencias del ejercicio

Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress. <https://git-scm.com/book/en/v2>

Git. (s. f.). Git documentation. <https://git-scm.com/docs>

GitHub. (s. f.). GitHub documentation. <https://docs.github.com/>

Ejercicio 2. El sistema gestor en un contenedor: Docker

Parte A. Investigación

1. ¿Qué es un contenedor y cómo se diferencia de una máquina virtual?

Un contenedor es una unidad aislada de ejecución que empaqueta una aplicación junto con los elementos que necesita para funcionar. Los contenedores utilizan mecanismos de aislamiento proporcionados por el sistema operativo anfitrión y comparten el kernel del sistema operativo, por lo que normalmente tienen un arranque más rápido y un tamaño menor que una máquina virtual.

Una máquina virtual, en cambio, virtualiza un equipo completo mediante un hipervisor. Cada máquina virtual ejecuta un sistema operativo invitado, además de sus aplicaciones. Esto proporciona un aislamiento fuerte y permite ejecutar sistemas operativos diferentes sobre el mismo hardware, pero implica mayor consumo de almacenamiento y memoria.

En términos de arranque, un contenedor suele iniciar en segundos o menos porque no necesita arrancar un sistema operativo completo. Una máquina virtual necesita iniciar su sistema operativo invitado. En tamaño, una imagen de contenedor puede ser relativamente pequeña, mientras que una máquina virtual suele incluir una imagen completa del sistema operativo. En aislamiento, ambos mecanismos separan procesos, pero las máquinas virtuales proporcionan una frontera basada en virtualización del hardware, mientras que los contenedores comparten el kernel del anfitrión.

Comparación

Aspecto	Contenedor	Máquina virtual
Arranque	Rápido; inicia el proceso o servicio de la aplicación.	Más lento; inicia un sistema operativo completo.
Tamaño	Generalmente menor.	Generalmente mayor porque incluye un sistema operativo invitado.
Kernel	Comparte el kernel del sistema anfitrión.	La VM ejecuta su propio kernel invitado.
Aislamiento	Aislamiento a nivel de procesos y recursos del sistema operativo.	Aislamiento mediante virtualización del sistema completo.
Uso típico	Servicios, microservicios, desarrollo y despliegue reproducible.	Sistemas completos, laboratorios y escenarios con distintos sistemas operativos.

2. Imagen, contenedor, volumen y puerto publicado

Imagen: Plantilla inmutable que contiene los archivos, dependencias, metadatos y configuración necesarios para crear contenedores. Una imagen puede tener varias capas y se utiliza como base para ejecutar una aplicación.

Contenedor: Instancia ejecutable de una imagen. Tiene su propio espacio de procesos, red y sistema de archivos escribible, según la configuración del runtime.

Volumen: Mecanismo administrado por Docker para almacenar datos fuera de la capa escribible del contenedor. Es especialmente importante para información que debe sobrevivir a la eliminación y recreación del contenedor.

Puerto publicado: Mapeo entre un puerto del equipo anfitrión y un puerto del contenedor. Permite que un servicio que escucha dentro del contenedor pueda ser alcanzado desde el anfitrión u otros clientes según las reglas de red.

3. ¿Por qué el volumen es indispensable y qué ocurre si no se declara?

Un volumen es indispensable cuando los datos generados por un servicio deben permanecer después de que el contenedor sea eliminado o reemplazado. En PostgreSQL, los archivos de datos se almacenan dentro del directorio de datos del servidor. Si ese directorio solamente pertenece a la capa escribible del contenedor y el contenedor se elimina, los datos almacenados allí se pierden junto con él.

Cuando se monta un volumen en la ruta de datos de PostgreSQL, los archivos persistentes quedan fuera del ciclo de vida del contenedor. Por ello es posible detener y eliminar el contenedor, crear otro contenedor que utilice el mismo volumen y encontrar nuevamente las bases de datos y tablas que existían anteriormente.

Por ejemplo, para PostgreSQL 16 puede utilizarse un volumen llamado pgdata montado en /var/lib/postgresql/data. El volumen no representa una copia temporal del contenedor, sino un espacio persistente administrado por Docker.

Configuración mínima relacionada con la práctica

```
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: practice1
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

La configuración anterior permite que PostgreSQL utilice una imagen conocida, publique el puerto 5432 y monte el volumen pgdata en el directorio donde PostgreSQL guarda sus datos.

Referencias del ejercicio

Docker. (s. f.). What is a container? Docker Documentation. <https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/>

Docker. (s. f.). Volumes. Docker Documentation. <https://docs.docker.com/engine/storage/volumes/>

Docker. (s. f.). Port publishing and mapping. Docker Documentation.
<https://docs.docker.com/engine/network/>

PostgreSQL Global Development Group. (s. f.). PostgreSQL documentation.
<https://www.postgresql.org/docs/>

Ejercicio 3. Investigación: qué es una base de datos

El objetivo de esta investigación es explicar los conceptos fundamentales de la Unidad Temática I: datos e información, características de las bases de datos, enfoque de archivos, usuarios, ciclo de vida, sistemas gestores de bases de datos, arquitecturas y modelos de datos.

1. Dato, información y base de datos

Un dato es una representación de un hecho, característica o valor que puede ser almacenado y procesado. Por sí solo puede tener un significado limitado, pero adquiere utilidad cuando se relaciona con un contexto. Por ejemplo, el valor 105 puede representar una cantidad, una edad, un peso o una medida dependiendo del contexto.

La información surge cuando los datos son procesados, organizados e interpretados de manera que proporcionan significado para un propósito. En un sistema de ventas, por ejemplo, los registros de productos y cantidades pueden transformarse en información sobre ventas diarias, productos más vendidos o ingresos.

Una base de datos es una colección organizada de datos relacionados que se administra de forma que pueda ser consultada, actualizada y compartida de manera controlada. Elmasri y Navathe (2016) explican las bases de datos como colecciones de datos relacionados con significado inherente y organizadas para servir a una aplicación. Silberschatz, Korth y Sudarshan (2019) presentan una base de datos como una colección de datos interrelacionados que modela información relevante de una empresa.

Las definiciones coinciden en tres ideas: existencia de datos relacionados, organización estructurada y propósito de representar información de un dominio. La diferencia principal está en el énfasis: Elmasri y Navathe destacan la colección estructurada con significado y la utilidad para aplicaciones, mientras que Silberschatz et al. resaltan las interrelaciones y el modelado de información de una organización.

2. Características de una base de datos

Integración: Los datos de distintas áreas pueden administrarse dentro de un conjunto coherente, reduciendo la necesidad de mantener copias independientes de la misma información.

Persistencia: Los datos se mantienen almacenados de manera que puedan ser utilizados posteriormente. El almacenamiento no depende de que una aplicación concreta esté ejecutándose.

Redundancia controlada: La duplicación innecesaria puede reducirse. Cuando existe redundancia por razones de rendimiento o disponibilidad, debe administrarse para evitar inconsistencias.

Integridad: El sistema debe conservar reglas que mantengan los datos válidos y coherentes, como claves primarias, claves foráneas, dominios y restricciones.

Independencia de datos: Permite modificar determinados aspectos del almacenamiento o del esquema sin obligar a modificar todas las aplicaciones que utilizan la información.

Seguridad: Incluye mecanismos de autenticación, autorización y control de acceso para impedir que usuarios no autorizados consulten o modifiquen información.

Concurrencia: Permite que varios usuarios o procesos trabajen simultáneamente manteniendo la consistencia de las operaciones mediante mecanismos de control de concurrencia.

Recuperación: Permite restaurar o recuperar información después de fallas mediante registros, respaldos y mecanismos de recuperación de transacciones.

3. Archivos frente a bases de datos

El enfoque tradicional basado en archivos almacena la información en archivos administrados directamente por aplicaciones. Cada programa puede definir su propia estructura y sus propios mecanismos para leer y actualizar los datos. Este modelo funcionó para sistemas pequeños, pero genera problemas cuando varias aplicaciones necesitan compartir información.

Entre sus principales dificultades están la redundancia, la inconsistencia, la dificultad para compartir información, la dependencia entre los programas y los formatos de los archivos, la limitada aplicación de restricciones globales y la complejidad de implementar seguridad y recuperación de forma uniforme.

La redundancia aparece cuando un mismo dato se almacena en múltiples archivos. Si una aplicación actualiza una copia y otra no, se produce inconsistencia. Además, un cambio en la estructura de un archivo puede obligar a modificar los programas que lo utilizan. Estas dificultades motivaron el desarrollo de los sistemas gestores de bases de datos, que centralizan mecanismos de almacenamiento, consulta, integridad, seguridad, concurrencia y recuperación.

4. Usuarios de una base de datos

Diseñador de bases de datos: Analiza los requisitos de información y diseña el esquema conceptual, lógico y, según el proceso, físico. Define entidades, atributos, relaciones y restricciones.

Administrador de bases de datos (DBA): Administra el sistema gestor, usuarios y permisos, seguridad, respaldo, recuperación, rendimiento, almacenamiento y disponibilidad.

Programador de aplicaciones: Construye programas que interactúan con la base de datos mediante lenguajes y APIs. Implementa consultas, operaciones de actualización y reglas de la aplicación.

Usuario final: Utiliza las aplicaciones o interfaces para consultar, registrar y modificar información de acuerdo con los permisos asignados.

Las responsabilidades y permisos deben separarse. Un usuario final no necesariamente debe tener permisos para modificar el esquema, mientras que el administrador requiere privilegios mayores para gestionar el sistema. El principio de mínimo privilegio recomienda conceder únicamente los permisos necesarios para realizar cada función.

5. Ciclo de vida de una base de datos

Etapa	Descripción	Entregable
-------	-------------	------------

1. Planificación	Se define el propósito, alcance, recursos, responsables y restricciones.	Plan de proyecto y alcance.
2. Análisis y recopilación de requisitos	Se identifica qué información necesita la organización, qué operaciones realizará y qué reglas debe cumplir.	Documento de requisitos.
3. Diseño conceptual	Se modela el dominio independientemente de la tecnología, identificando entidades, atributos y relaciones.	Modelo conceptual o diagrama entidad-relación.
4. Diseño lógico	El modelo conceptual se transforma al modelo lógico seleccionado, por ejemplo el modelo relacional.	Esquema lógico, tablas, claves y restricciones.
5. Diseño físico	Se decide cómo almacenar y acceder eficientemente a los datos según el SGBD.	Esquema físico, índices y parámetros de almacenamiento.
6. Implementación	Se crea la base de datos, tablas, restricciones, usuarios, procedimientos y demás componentes necesarios.	Base de datos implementada y scripts.
7. Pruebas y validación	Se comprueba que los datos, consultas, restricciones, seguridad y operaciones cumplen los requisitos.	Resultados de pruebas.
8. Operación y mantenimiento	Se monitorea, respalda, optimiza y modifica la base de datos conforme cambian las necesidades.	Respaldos, reportes de rendimiento y nuevas versiones.

6. Sistema gestor de bases de datos

Un sistema gestor de bases de datos (SGBD o DBMS) es el software que permite definir, crear, manipular, proteger y administrar bases de datos. Actúa como intermediario entre las aplicaciones y los datos almacenados.

Entre sus componentes y funciones se encuentran el procesador de consultas, el optimizador, el administrador de almacenamiento, el administrador de transacciones, el control de concurrencia, los mecanismos de recuperación, el catálogo o diccionario de datos y los componentes de seguridad.

Los principales lenguajes y categorías de operaciones son:

DDL (Data Definition Language): Define o modifica estructuras, como CREATE, ALTER y DROP.

DML (Data Manipulation Language): Consulta y modifica datos, incluyendo SELECT, INSERT, UPDATE y DELETE.

DCL (Data Control Language): Gestiona privilegios, por ejemplo GRANT y REVOKE.

Control de transacciones: Administra unidades de trabajo mediante operaciones como COMMIT y ROLLBACK.

Los SGBD pueden clasificarse de distintas formas. Por ejemplo, existen sistemas relacionales, orientados a documentos, clave-valor, columnares, de grafos y otros enfoques especializados. En un SGBD relacional, los datos se representan principalmente mediante relaciones o tablas y se utilizan claves y restricciones para expresar asociaciones e integridad.

7. Sistema de base de datos: arquitectura de tres niveles

La arquitectura de tres niveles, asociada al modelo ANSI/SPARC, separa la representación externa, conceptual e interna de los datos.

Nivel externo: Representa las vistas particulares que diferentes usuarios o aplicaciones necesitan. Una organización puede tener múltiples vistas sobre la misma base de datos.

Nivel conceptual: Describe la estructura lógica global de la base de datos: entidades, relaciones, restricciones y tipos de información, sin entrar en detalles del almacenamiento físico.

Nivel interno: Describe cómo se almacenan físicamente los datos: estructuras de archivos, índices, páginas, organización de almacenamiento y otros mecanismos internos del SGBD.

La separación de niveles favorece la independencia de datos. La independencia física busca que cambios en el almacenamiento interno no obliguen a cambiar el esquema conceptual. La independencia lógica busca que ciertos cambios en el esquema conceptual puedan realizarse sin modificar todas las vistas externas.

Arquitecturas centralizada y cliente-servidor

En una arquitectura centralizada, los principales componentes del SGBD y los datos se encuentran en un sistema central. En una arquitectura cliente-servidor, los clientes solicitan servicios al servidor de base de datos. El servidor procesa consultas, administra los datos y aplica mecanismos de seguridad y concurrencia, mientras que el cliente proporciona una interfaz o lógica adicional.

La arquitectura cliente-servidor permite distribuir responsabilidades y facilita que múltiples aplicaciones se conecten a un mismo servidor de datos. En sistemas actuales también pueden utilizarse arquitecturas de varias capas, donde presentación, lógica de negocio y acceso a datos están separados.

8. Modelos de datos: importancia y clasificación

Un modelo de datos proporciona conceptos para describir la estructura de los datos, las relaciones entre ellos, las restricciones y, dependiendo del modelo, las operaciones que pueden realizarse. Es importante porque permite representar los requisitos de un sistema antes de implementar físicamente la base de datos y proporciona un lenguaje común entre usuarios, analistas y desarrolladores.

Por nivel de abstracción, pueden distinguirse modelos conceptuales, lógicos y físicos. El modelo conceptual representa el dominio de forma independiente de un SGBD específico; el lógico transforma esa descripción a un modelo de datos concreto, como el relacional; y el físico describe cómo se implementará el almacenamiento.

Por estructura, existen modelos como el jerárquico, de red, relacional, orientado a objetos y diversos modelos NoSQL. El modelo relacional representa datos mediante relaciones y utiliza claves y restricciones. Los modelos documentales representan información como documentos, mientras que los modelos de grafos representan entidades y relaciones como nodos y aristas. La elección depende de los requisitos del sistema, los tipos de consultas, la consistencia requerida y la forma de los datos.

Conclusión

Una base de datos no consiste únicamente en un conjunto de archivos: forma parte de un sistema compuesto por datos, reglas, usuarios, aplicaciones y un sistema gestor que proporciona mecanismos para administrarlos. Las características de integración, persistencia, integridad, seguridad, concurrencia y recuperación permiten resolver limitaciones que aparecen cuando la información se maneja exclusivamente mediante archivos. El estudio de las arquitecturas y modelos de datos permite pasar de los requisitos de un problema real a una representación organizada que posteriormente puede implementarse en un SGBD.

Referencias del ejercicio

- Connolly, T., & Begg, C. (2015). Database systems: A practical approach to design, implementation, and management (6th ed.). Pearson.
- Elmasri, R., & Navathe, S. B. (2016). Fundamentals of database systems (7th ed.). Pearson.
- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database system concepts (7th ed.). McGraw-Hill.
- Coronel, C., & Morris, S. (2019). Database systems: Design, implementation, & management (13th ed.). Cengage.

Ejercicio 4. Estado del arte: tres artículos científicos

Para este ejercicio se seleccionaron tres artículos científicos relacionados con el uso de bases de datos y publicados en revistas académicas con revisión por pares y DOI. Los trabajos se centran en problemas actuales de gestión, optimización y análisis de datos relacionales. Los tres artículos son posteriores a 2018, por lo que cumplen con la preferencia temporal indicada en la práctica.

Artículo 1. Data dependencies for query optimization: a survey

1. Cita en formato APA 7

Kossmann, J., Papenbrock, T., & Naumann, F. (2022). Data dependencies for query optimization: A survey. *The VLDB Journal*, 31(1), 1–22. <https://doi.org/10.1007/s00778-021-00676-3>

2. Problema que aborda

El artículo analiza el uso de dependencias existentes en los datos, como dependencias funcionales, de inclusión y de orden, para mejorar la optimización de consultas. El problema surge porque los optimizadores necesitan información sobre los datos para seleccionar planes de ejecución eficientes, pero muchas dependencias útiles no están disponibles explícitamente.

3. Método o propuesta de los autores

Se presenta una revisión de la literatura sobre dependencias de datos y su utilización en la optimización de consultas. Los autores organizan los distintos tipos de dependencias, explican cómo pueden descubrirse o mantenerse y analizan de qué manera pueden utilizarse durante la optimización.

4. Resultado principal

El trabajo sintetiza el estado del conocimiento sobre las dependencias de datos y muestra que estas pueden aportar información relevante para elegir mejores estrategias de ejecución. También identifica dificultades prácticas relacionadas con descubrir y mantener dichas dependencias.

5. Relación con la Unidad Temática I

Se relaciona directamente con el tema de sistemas gestores de bases de datos y con las características de integridad e independencia de datos. También se vincula con el procesamiento y optimización de consultas, ya que el SGBD necesita administrar información estructurada y metadatos para ejecutar consultas de manera eficiente.

6. Aporte para el proyecto del curso

Este artículo ayuda a comprender que el diseño de una base de datos no termina al crear tablas. Las relaciones y restricciones existentes entre los datos también pueden proporcionar información que un SGBD utiliza para optimizar consultas. Para un proyecto de curso, esto refuerza la importancia de definir correctamente las claves, restricciones y relaciones.

Artículo 2. Efficient distributed discovery of bidirectional order dependencies

1. Cita en formato APA 7

Schmidl, S., & Papenbrock, T. (2022). Efficient distributed discovery of bidirectional order dependencies. *The VLDB Journal*, 31, 49–74. <https://doi.org/10.1007/s00778-021-00683-4>

2. Problema que aborda

El artículo estudia la detección de dependencias de orden bidireccionales en tablas relacionales. Estas dependencias permiten identificar relaciones entre órdenes de atributos, pero descubrir todas las dependencias mínimas puede ser costoso cuando el número de atributos aumenta.

3. Método o propuesta de los autores

Los autores proponen DISTOD, un algoritmo distribuido para descubrir dependencias de orden bidireccionales. El método distribuye la búsqueda y validación entre varias máquinas y utiliza estrategias de poda, generación paralela de candidatos y administración dinámica de recursos.

4. Resultado principal

La evaluación reporta que DISTOD puede superar a algoritmos anteriores de descubrimiento de dependencias de orden, alcanzando mejoras de varios órdenes de magnitud en determinados conjuntos de datos. El enfoque permite trabajar con conjuntos de datos mayores aprovechando recursos distribuidos.

5. Relación con la Unidad Temática I

Se relaciona con modelos de datos relacionales, dependencias entre atributos, sistemas gestores de bases de datos y procesamiento de información. También muestra cómo las propiedades estructurales de una relación pueden aprovecharse para tareas de gestión y optimización.

6. Aporte para el proyecto del curso

El artículo muestra la importancia de analizar las relaciones existentes entre atributos y cómo los problemas de descubrimiento de información pueden requerir técnicas de procesamiento distribuido. Para un proyecto de bases de datos, permite valorar la diferencia entre simplemente almacenar información y analizar las propiedades de los datos para mejorar su procesamiento.

Artículo 3. Robust Cardinality: a novel approach for cardinality prediction in SQL queries

1. Cita en formato APA 7

Praciano, F. D. B. S., Amora, P. R. P., Abreu, I. C., Pereira, F. L. F., & Machado, J. C. (2021). Robust Cardinality: A novel approach for cardinality prediction in SQL queries. *Journal of the Brazilian Computer Society*, 27, Article 11. <https://doi.org/10.1186/s13173-021-00115-9>

2. Problema que aborda

Los sistemas gestores de bases de datos necesitan estimar cuántas filas producirá una operación o consulta para elegir planes de ejecución. Los errores en estas estimaciones pueden provocar que el optimizador seleccione una estrategia poco eficiente.

3. Método o propuesta de los autores

Los autores proponen Robust Cardinality, un método basado en aprendizaje automático para realizar estimaciones de cardinalidad. El enfoque utiliza Gradient Boosted Decision Trees y contempla un intervalo de confianza para detectar estimaciones que podrían ser poco fiables.

4. Resultado principal

En las pruebas realizadas con PostgreSQL, el método redujo el error de estimación y produjo mejoras en el tiempo de ejecución de consultas. El artículo reporta una reducción del 3 % en el tiempo de ejecución después de reducir cuatro veces el error de estimación en el conjunto de pruebas descrito.

5. Relación con la Unidad Temática I

El trabajo se relaciona directamente con los sistemas gestores de bases de datos, el lenguaje SQL y la optimización de consultas. También permite observar cómo técnicas de aprendizaje automático pueden incorporarse a componentes internos de un SGBD.

6. Aporte para el proyecto del curso

Este artículo permite entender por qué el SGBD necesita tomar decisiones internas para ejecutar consultas. Aunque el proyecto inicial puede ser pequeño, conocer el papel de las estimaciones de cardinalidad ayuda a comprender por qué el diseño de tablas, relaciones e índices influye posteriormente en el rendimiento.

Comparación de los tres artículos

Los tres artículos estudian problemas relacionados con el procesamiento eficiente de información en sistemas de bases de datos, pero lo hacen desde perspectivas diferentes. Kossmann et al. presentan una revisión de dependencias de datos y su utilidad para optimizar consultas. Schmidl y Papenbrock se concentran en el descubrimiento distribuido de dependencias de orden. Praciano et al. estudian específicamente la estimación de cardinalidad mediante aprendizaje automático.

En común, los tres trabajos muestran que un SGBD necesita información adicional sobre los datos para procesarlos eficientemente. No basta con almacenar registros: el sistema debe analizar propiedades de los datos, estimar costos y seleccionar estrategias de ejecución. También comparten la preocupación por trabajar con conjuntos de datos que pueden crecer y por reducir el costo computacional de los procesos.

La principal diferencia está en el tipo de problema. El primer artículo tiene carácter de revisión y organiza conocimiento existente sobre dependencias. El segundo propone un algoritmo distribuido para descubrir dependencias de orden. El tercero propone un estimador basado en aprendizaje automático para mejorar una tarea específica del optimizador.

A partir de la lectura se identifica como problema abierto la necesidad de que los sistemas gestores puedan obtener información fiable sobre los datos con un costo razonable y mantenerla actualizada cuando los datos cambian. Esto es relevante porque una estimación o dependencia que era válida para un conjunto de datos puede dejar de representar correctamente el estado actual conforme se insertan, modifican o eliminan registros.

Referencias del ejercicio

- Kossmann, J., Papenbrock, T., & Naumann, F. (2022). Data dependencies for query optimization: A survey. *The VLDB Journal*, 31(1), 1–22. <https://doi.org/10.1007/s00778-021-00676-3>
- Schmidl, S., & Papenbrock, T. (2022). Efficient distributed discovery of bidirectional order dependencies. *The VLDB Journal*, 31, 49–74. <https://doi.org/10.1007/s00778-021-00683-4>
- Praciano, F. D. B. S., Amora, P. R. P., Abreu, I. C., Pereira, F. L. F., & Machado, J. C. (2021). Robust Cardinality: A novel approach for cardinality prediction in SQL queries. *Journal of the Brazilian Computer Society*, 27, Article 11. <https://doi.org/10.1186/s13173-021-00115-9>

Referencias generales

- Chacon, S., & Straub, B. (2014). *Pro Git* (2nd ed.). Apress. <https://git-scm.com/book/en/v2>
- Connolly, T., & Begg, C. (2015). *Database systems: A practical approach to design, implementation, and management* (6th ed.). Pearson.
- Docker. (s. f.). Docker documentation. <https://docs.docker.com/>
- Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7th ed.). Pearson.
- Git. (s. f.). Git documentation. <https://git-scm.com/docs>
- GitHub. (s. f.). GitHub documentation. <https://docs.github.com/>
- PostgreSQL Global Development Group. (s. f.). PostgreSQL documentation. <https://www.postgresql.org/docs/>
- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database system concepts* (7th ed.). McGraw-Hill.